OpenMP Practical Lab Report

Benchmark tests executed on 16-core AMD CPU, 16GB of RAM, on Arch Linux. Worked together with Tomoki Mengel.

1 Calculating Pi

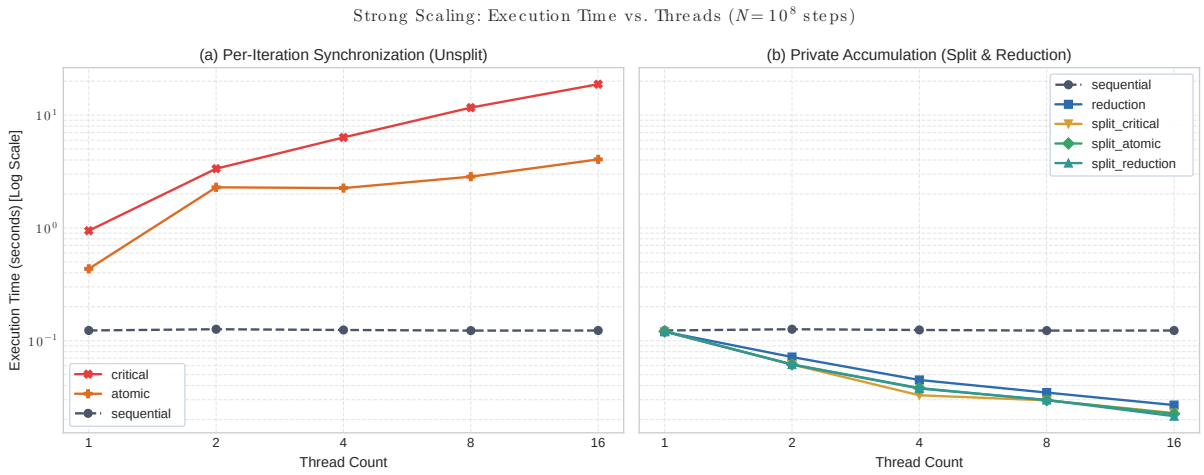


Figure 1: **Execution Time vs. Thread Count**

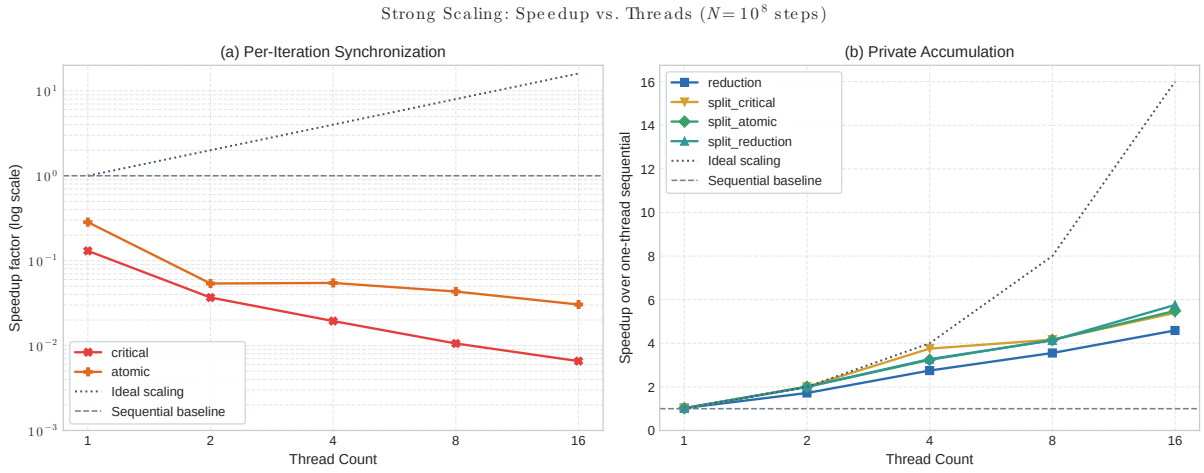


Figure 2: **Speedup vs. Thread Count**

1.1 Performance analysis

I have displayed the differrent methods on the two graphs. Both graphs show the runtime of the benchmarks by thread count. The first grahp shows an increase of the runtime with the increase of the number of threads. It's because the synchronisation between threads happen in each loop, costing a lot of performance. In the other cases, where sequential and the manual splits are displayed, we can see a decline in execution time, which is because the calculation is actually distributed between the threads, and the costy synchronisation is executed only as many time as many threads there are (16 times in case of 16 threads for example).

2 Matrix-Vector Calculation

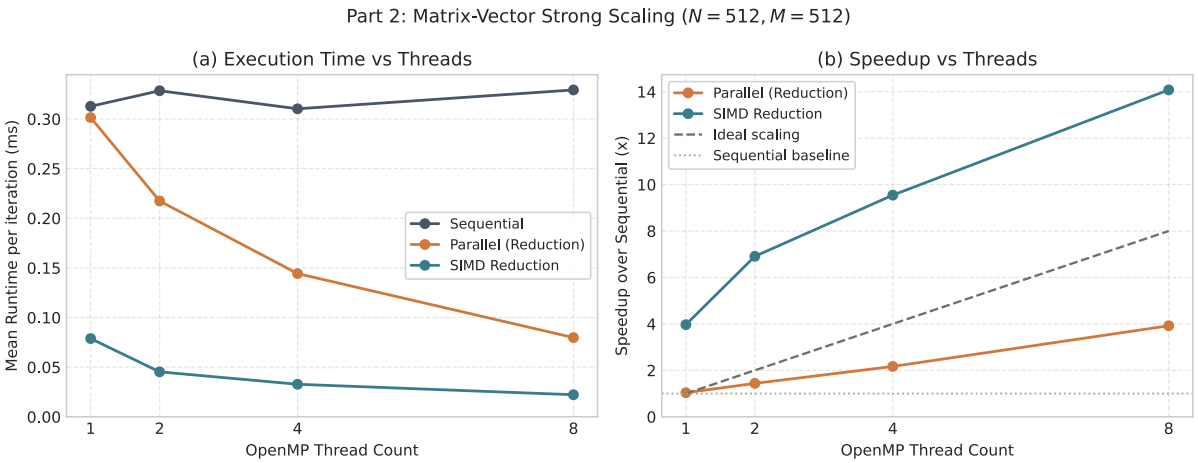


Figure 3: **Matrix-Vector Strong Scaling (Execution Time and Speedup)**

2.1 Performance analysis

The graphs above show the performance differences between different methods, showing execution time and speedup as the thread count increases. The sequential version remains constant as expected. In the parallel reduction method, the loop over the rows is distributed across thraeds using OpenMP reduction, which makes runtime faster with a speedup of 3.9x with 8 threads. The SIMD method improves performance too by vectorising the inner row multiplications. Parallelising the floating point operations earns additional speedup, resulting in a 14x speedup.

3 Fibonacci List

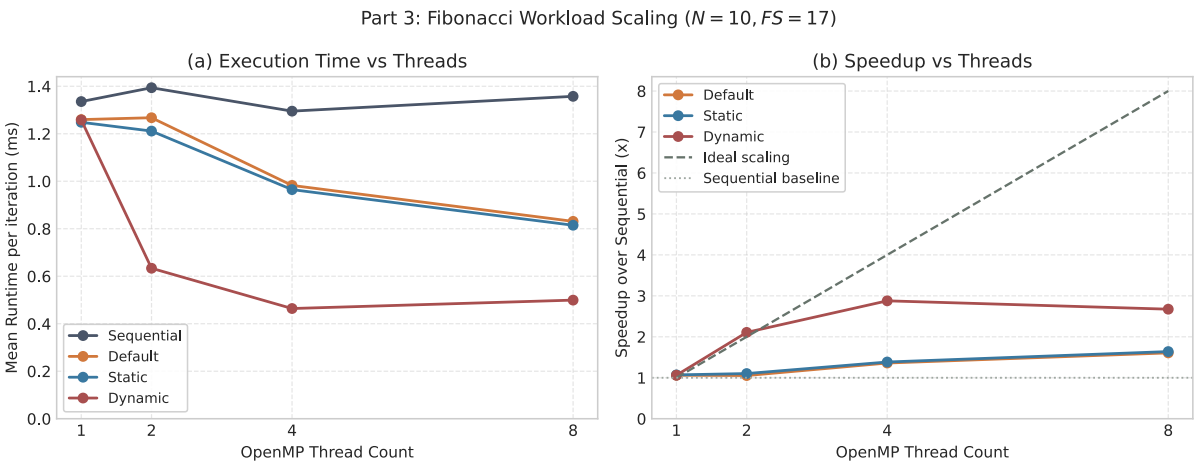


Figure 4: **Fibonacci Workload Scaling by Scheduling Strategy (Execution Time and Speedup)**

3.1 Performance analysis

We can see the different scheduling strategies on the diagrams, comparing their execution time and speedup across thread counts. The recursive Fibonacci calculation is not balanced, because computing higher Fibonacci numbers requires (exponentially) more operations, resulting in varying wrokload. In the static scheduling strategy, iterations are divided into fixed chunks between threads, so threads assigned to larger numbers do most of the calculation while other threads finish early and stay idle, leading to a modest speedup of only about 1.6x at 8 threads. However, the dynamic scheduling strategy allocates tasks to threads on demand as soon as they become available. This enables distributing the workload better, leading to much better thread utilisation and achieving a speedup of around 2.9x at 4 threads.

We had problems running the simulation, because initially we didn't understand why we don't see a difference between different strategies. As it turns out, the default configuration only worked on 7-8 numbers, so it wouldn't really make a difference in performance, since my computer has 16 threads. It means, that whatever the strategy is, the computation takes as long as the greates fibonacci number is calculated on the one of the threads. After realising, I adjusted the number of threads to a lower number, and also made the number of elements to be calculated higher, so the difference betweeen each strategy is more visible.